# EngramTrace: A Non-Parametric Long-Term Memory Architecture for LLM Agents via Hierarchical DOM-Based Retrieval and Online Knowledge Consolidation

Taehyun Park[1]

[1]College of Letters & Science, University of Wisconsin–Madison, tpark45@wisc.edu,
https://github.com/iamtaehyunpark/EngramTrace

**Abstract**

Large Language Models (LLMs) exhibit powerful parametric knowledge but lack persistent, updateable memory across interactions. Existing Retrieval-Augmented Generation (RAG) systems partially address this gap yet suffer from two fundamental limitations: semantic fragmentation caused by arbitrary chunking of documents into isolated embedding vectors, and static indexing that cannot incorporate new knowledge without full re-indexing. Graph-based approaches offer richer associative structures but introduce significant infrastructure overhead, rigid schema constraints, and representations that are foreign to the native processing patterns of transformer-based LLMs.

This report presents **EngramTrace**, a non-parametric long-term memory architecture for LLM agents that addresses these limitations by maintaining a *living HTML document* as its knowledge base, wherein the Document Object Model (DOM) serves as an implicit, schema-free virtual graph. This design exploits the inherent HTML literacy of LLMs acquired during pre-training on large web corpora, enabling native hierarchical reasoning over the knowledge structure without dedicated graph infrastructure. EngramTrace introduces three core contributions: (1) a *Hierarchical Embedding Summation* scheme in which leaf-node vectors are blended with ancestral context signals to resolve the semantic isolation problem of flat retrieval; (2) an *online memory consolidation cycle* enabling continuous, non-destructive knowledge base updates from conversational interaction without model retraining; and (3) *batch processing of inter-node connectivity*—since nodes exist as textual content embedded within a single overarching HTML structure, an LLM can process all node relationships and their linkages in a single request, enabling efficient, global synchronization of knowledge connectivity without sequential node-by-node operations. The system is evaluated on the MuSiQue multi-hop QA benchmark as a preliminary test of retrieval quality.

# Contents

**1 Introduction** **3**

**2 Related Work** **4**
2.1 Retrieval-Augmented Generation . . . 4
2.2 Graph-Augmented RAG . . . 4
2.3 HTML as a Knowledge Format for LLMs . . . 4
2.4 Long-Term Conversational Memory . . . 5

**3 System Overview** **5**

**4 Methodology** **6**
4.1 The HTML Knowledge Base: Structure and Initialization . . . 6
4.1.1 Structural Foundation . . . 6
4.1.2 Atomization . . . 6
4.1.3 Hierarchical Embedding Computation . . . 6
4.2 Hierarchical Embedding Summation: Resolving Semantic Isolation . . . 6
4.3 Retrieval: The Ecphory Mechanism . . . 7
4.3.1 Mode A: Interaction-Blended Vectorial Retrieval . . . 7
4.3.2 Mode B: Lightweight Keyword Search . . . 7
4.3.3 Trace Set Assembly . . . 7
4.4 Stage Drift Detection and Focus Regulation . . . 7
4.5 Memory Consolidation: Online Knowledge Base Updates . . . 8
4.6 Homeostatic Day System . . . 8

**5 The DOM as a Virtual Graph** **8**
5.1 Graph-Theoretic Properties . . . 8
5.2 Advantages Over Explicit Graph Databases . . . 9
5.3 Limitations of the Virtual Graph Approach . . . 9

**6 Evaluation** **10**
6.1 Benchmark: MuSiQue . . . 10
6.2 Experimental Setup . . . 10
6.3 Results . . . 10

**7 Qualitative Analysis** **11**
7.1 Novel Concept Knowledge Base . . . 11
7.2 Technical Documentation Assistant . . . 11

**8 Discussion** **12**
8.1 Positioning Relative to Explicit Graph-Based Systems . . . 12
8.2 Online Consolidation vs. Static Indexing . . . 12
8.3 Limitations . . . 12
8.4 Future Directions . . . 12

**9 Conclusion** **13**

# 1 Introduction

The capability of modern LLMs to encode and retrieve vast stores of world knowledge within their parameters is well established [1, 15]. Yet despite this breadth, LLM-powered agents face a persistent limitation: they possess no robust mechanism for persistent, updateable, non-parametric memory across interactions. Each conversation begins from a fixed parametric snapshot; knowledge acquired during deployment cannot be integrated without expensive retraining. For agents deployed in knowledge-intensive, evolving domains such as technical documentation management, personal assistants, and research collaboration, this is a fundamental architectural deficiency.

Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for addressing this limitation [2, 10]. However, standard RAG systems treat the knowledge corpus as a flat collection of independently embedded text chunks—what we call *brutal chunking*. This severs contextual relationships inherent in document structure and provides no mechanism for representing associative relationships between knowledge units. These limitations become most visible in multi-hop reasoning tasks, where answering a query requires traversing several associative steps across knowledge elements.

Graph-based RAG architectures have been proposed to address the associativity problem. HippoRAG [3] constructs an open knowledge graph for associative multi-hop retrieval, demonstrating improvements of up to 20% on multi-hop QA benchmarks. GraphRAG [4] uses community detection over entity graphs for global summarization. Zep/Graphiti [5] maintains a temporally-aware knowledge graph with bi-temporal fact invalidation. These systems demonstrate convincingly that graph-structured knowledge representations outperform flat vector stores for complex retrieval tasks.

However, graph-based systems introduce their own class of limitations: they require dedicated graph database infrastructure (Neo4j in Zep's case), impose schema constraints through the entity-relation triple paradigm, and produce representations structurally alien to transformer-based LLMs. The impedance mismatch between the graph representation and the LLM's native input modality necessitates a translation layer that may introduce information loss.

The key insight motivating this work is that **transformer-based LLMs are natively proficient at reading and reasoning over HTML**. This proficiency arises from the massive quantity of HTML encountered during pre-training on web corpora [6]. The Document Object Model provides a native, human-readable, schema-free hierarchical structure where parent-child relationships encode contextual containment, heading-body relationships encode topic hierarchy, and sibling relationships encode conceptual adjacency—all without any specialized translation layer.

EngramTrace maintains the agent's entire knowledge base as a single, mutable HTML document. The DOM functions as a *virtual graph*—providing hierarchical associative structure without a dedicated graph database, without schema constraints, and in a format LLMs process natively. Knowledge is continuously integrated from conversational interactions through an online consolidation cycle, and retrieval operates through a dual-mode mechanism combining semantic vector search with hierarchical context inheritance.

The primary contributions are:

1. **The DOM-as-Virtual-Graph paradigm:** HTML's DOM as an implicit, schema-free graph structure for LLM memory, natively interpretable by both transformer models and human operators.
2. **Hierarchical Embedding Summation:** A retrieval-time embedding scheme blending each node's local vector with its ancestral context signal, resolving semantic isolation in

flat retrieval.

3. **Batch processing of inter-node connectivity:** Since nodes exist as textual content embedded within a single overarching HTML structure, an LLM can process all node relationships and their linkages in a single request, enabling efficient global synchronization of knowledge connectivity without sequential node-by-node operations.
4. **Online Memory Consolidation:** A continuous, non-destructive knowledge update cycle synthesizing episodic interactions into structured HTML mutations of the persistent knowledge base.

# 2 Related Work

## 2.1 Retrieval-Augmented Generation

RAG, introduced by Lewis et al. [2], provides LLMs with access to external, updateable knowledge through a retriever that selects relevant passages and a reader LLM that generates conditioned responses. The dominant implementation encodes document chunks as dense vectors and retrieves via approximate nearest-neighbor search [10, 12]. Standard RAG fails when relevant knowledge is distributed across multiple passages requiring joint interpretation, and when the knowledge base evolves over time—both regimes EngramTrace targets.

## 2.2 Graph-Augmented RAG

HippoRAG [3] draws on the hippocampal memory indexing theory [9, 18] to construct a schemaless open knowledge graph via LLM-based open information extraction, retrieving through Personalized PageRank [16] seeded at query-relevant entity nodes. While powerful for multi-hop retrieval, the KG triple representation requires translation back to text for LLM consumption and demands LLM inference at indexing time.

GraphRAG [4] constructs hierarchical community summaries using the Leiden algorithm, enabling global query-focused summarization. Its offline summarization requires full re-indexing when new information arrives, limiting applicability to dynamic knowledge bases.

Zep/Graphiti [5] maintains a temporally-aware KG with a bi-temporal model distinguishing the chronological event timeline ($T$) from the transactional ingestion timeline ($T'$), enabling dynamic invalidation of stale facts. Zep demonstrates up to 18.5% accuracy improvement over full-context baselines with 90% latency reduction. However, it relies on Neo4j as its backend and LLM-based entity extraction at ingestion time.

EngramTrace replaces the explicit graph database with the HTML DOM as an implicit graph, eliminating schema constraints, infrastructure overhead, and the LLM-to-graph translation mismatch.

## 2.3 HTML as a Knowledge Format for LLMs

HtmlRAG [6] demonstrates that preserving HTML structure during RAG—rather than converting to plain text—yields significant improvements across six QA benchmarks. The structural and semantic information encoded in HTML tags is lost during plain-text conversion, while LLMs possess robust native HTML comprehension from pre-training. EngramTrace extends this insight from the retrieval context into the *memory architecture* context: rather than using HTML for retrieved web documents, we use a curated, maintained HTML document as the agent's persistent knowledge base.

### 2.4 Long-Term Conversational Memory

MemGPT [8] introduced LLMs as operating systems managing hierarchical memory with self-directed memory operations. While influential, MemGPT relies on the LLM itself to manage memory at inference time, introducing latency and complexity. Zep [5] and related work have demonstrated that dedicated memory architectures operating outside the LLM inference path yield superior performance with dramatically reduced latency. EngramTrace follows this philosophy: memory management is handled by a dedicated architecture asynchronously from LLM inference.

## 3 System Overview

EngramTrace operates through four principal stages: **Initialization** (atomization of seed knowledge into structured HTML), **In-Stage Retrieval and Inference** (Ecphory-based context retrieval for response generation), **Consolidation** (online synthesis of episodic interactions into persistent knowledge updates), and **Homeostasis** (periodic restructuring and deduplication of the knowledge base).

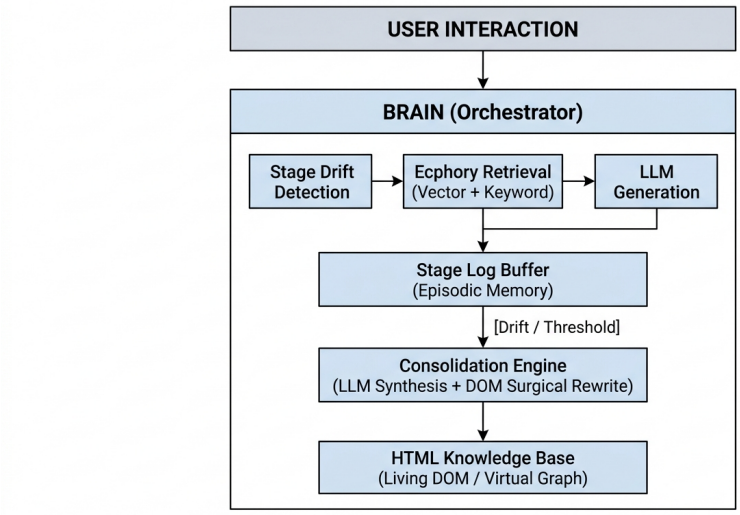



Figure 1: EngramTrace system pipeline. The Brain orchestrator routes each interaction through drift detection, Ecphory retrieval, and LLM generation. Episodic interactions accumulate in the Stage Log until consolidation is triggered, at which point the knowledge base undergoes a surgical DOM rewrite.

The system has three distinctive core cognitive components: the Brain orchestrator, the retrieval trace manager (EngramTrace session manager), and the MemoryManager. The LLM layer uses Google Gemini Flash-Lite for generation and Gemini Embedding-001 (256-dimensional) for embedding computation, though the architecture is model-agnostic. Knowledge base parsing and DOM manipulation are handled via BeautifulSoup4 [13].

The frontend provides a Chat Console for interaction with live DOM graph visualization.

# 4 Methodology

## 4.1 The HTML Knowledge Base: Structure and Initialization

### 4.1.1 Structural Foundation

The EngramTrace knowledge base (KB) is a single HTML document $\mathcal{K}$ maintained persistently on disk. Semantically, $\mathcal{K}$ is a hierarchically structured repository of knowledge units organized by topic, subtopic, and detail level encoded as HTML heading and block elements. The parsed DOM of $\mathcal{K}$ constitutes a tree $\mathcal{T} = (\mathcal{N}, \mathcal{E})$ where $\mathcal{N}$ is the set of DOM nodes and $\mathcal{E}$ is the set of parent-child edges.

### 4.1.2 Atomization

The system is initialized through *Atomization*: unstructured seed knowledge is converted into a structured HTML document by an LLM prompted to identify the hierarchical conceptual structure of the input and render it as well-formed HTML with semantically meaningful tags (`<h1>` for top-level topics, `<h2>`/`<h3>` for subtopics, `<p>` for factual assertions, `<ul>`/`<li>` for enumerable properties).

Each node $n \in \mathcal{N}$ is assigned a deterministic identifier via SHA-256 hashing of its text content:

$$\mathrm{id}(n) = \mathrm{SHA256}(\mathrm{content}(n)) \tag{1}$$

This deterministic mapping enables stable tracking of nodes across consolidation cycles, allowing the system to identify which nodes require modification or removal when synchronizing the knowledge base.

### 4.1.3 Hierarchical Embedding Computation

Following atomization, embeddings are computed for all nodes using a batch procedure with top-down memoization. Let $\mathbf{v}_n^{\text{local}}$ denote the raw embedding of node $n$'s text content. The hierarchical embedding $\mathbf{v}_n$ is a weighted combination of its local embedding and its parent's hierarchical embedding:

$$\mathbf{v}_n = \alpha \cdot \mathbf{v}_n^{\text{local}} + (1 - \alpha) \cdot \mathbf{v}_{\text{parent}(n)} \tag{2}$$

where $\alpha = 0.7$ is the blending coefficient (a working default) and the root node's hierarchical embedding equals its local embedding. All hierarchical embeddings are stored in a `numpy` array indexed by node identifier for efficient cosine similarity computation.

## 4.2 Hierarchical Embedding Summation: Resolving Semantic Isolation

A core limitation of flat retrieval is *semantic isolation*: a leaf node embedded in isolation captures only local text content, disregarding the hierarchical context that determines its meaning. For example, a paragraph about a specific API method carries different meaning depending on whether its parent section concerns authentication or data formatting, yet flat retrieval would assign nearly identical embeddings if both paragraphs share surface vocabulary.

The Hierarchical Embedding Summation scheme propagates contextual signal from ancestor nodes down to descendants. Through recursive application of the blending equation, the hierarchical embedding of a leaf node at depth $d$ implicitly encodes contributions from all ancestors along the path from root to $n$, with contributions decaying geometrically with distance:

$$\mathbf{v}_n = \alpha^d \mathbf{v}_n^{\text{local}} + \alpha^{d-1}(1 - \alpha)\mathbf{v}_{\text{parent}(n)}^{\text{local}} + \cdots + (1 - \alpha)\mathbf{v}_{\text{root}}^{\text{local}} \tag{3}$$

We term this property *ancestral awareness*. The blending coefficient $\alpha = 0.7$ reflects the hypothesis that local content should dominate while still receiving meaningful ancestral signal.

## 4.3 Retrieval: The Ecphory Mechanism

We adopt the neuropsychological term *Ecphory*—the process by which a partial cue triggers reconstruction of a stored memory trace—to describe EngramTrace's retrieval mechanism. Ecphory is implemented as a dual-mode strategy.

### 4.3.1 Mode A: Interaction-Blended Vectorial Retrieval

To maintain *topic trajectory*—biasing retrieval toward the current conversational context—the query embedding is blended with recent interaction history. For incoming query $q$ with embedding $\mathbf{q}$:

$$\tilde{\mathbf{q}} = \beta \cdot \mathbf{q} + (1 - \beta) \cdot \bar{\mathbf{h}} \tag{4}$$

where $\bar{\mathbf{h}}$ is the mean embedding of the previous $k = 3$ query-answer pairs and $\beta = 0.7$. Retrieval computes cosine similarity against all hierarchical node embeddings:

$$\text{score}_A(n) = \frac{\tilde{\mathbf{q}} \cdot \mathbf{v}_n}{\|\tilde{\mathbf{q}}\| \|\mathbf{v}_n\|} \tag{5}$$

Nodes with a cosine similarity exceeding the threshold (0.80) are retrieved as candidates relevant to the query and used to generate a response, with a top-3 fallback when the candidate pool is sparse. Retrieved nodes' full DOM subtrees are assembled into the retrieval context $\mathcal{C}$.

### 4.3.2 Mode B: Lightweight Keyword Search

Dense vector retrieval can miss precise lexical matches for domain-specific terminology, proper nouns, and technical identifiers whose meaning may not be reliably captured in pre-trained embedding spaces. Mode B addresses this through a hand-crafted keyword search: query terms are tokenized and stop-word-filtered, and the resulting term set is matched against node content through direct string matching with basic normalization. This is intentionally lightweight—keyword search in EngramTrace is a complementary signal rather than a primary retrieval axis.

Mode B is activated during stage transitions (Section 4.4), capturing lexical signals that may be washed out in the dense embedding space during topic changes.

### 4.3.3 Trace Set Assembly

The final retrieval context $\mathcal{C}$ is assembled from Mode A and Mode B candidates. For each retrieved node $n$, the system includes both the node's content and its *ancestral path*—the sequence of ancestor nodes from root to $n$—preserving hierarchical context. Retrieved nodes are maintained in a trace set persisting across the current conversational stage, enabling cumulative context assembly.

## 4.4 Stage Drift Detection and Focus Regulation

EngramTrace partitions the conversational stream into *stages*: contiguous spans sharing a coherent topical focus. Stage boundaries are detected by monitoring the running cosine similarity between recent interactions and the established stage context.

Let $\bar{\mathbf{s}}$ denote the rolling mean embedding of the current stage's last $k$ interaction pairs. For each new query $q$:

$$\text{drift}(q) = \frac{\mathbf{q} \cdot \bar{\mathbf{s}}}{\|\mathbf{q}\| \|\bar{\mathbf{s}}\|} \tag{6}$$

Stage drift is declared when $\text{drift}(q) < \theta_{\text{drift}}$ (default 0.83, empirically tuned). Upon detection, the current stage closes, consolidation is triggered, and a new stage initializes.

The system also supports `no_search` (restrict context to current trace set) and `no_memorize` (skip stage log write) inference control modes.

### 4.5 Memory Consolidation: Online Knowledge Base Updates

Consolidation is the process by which episodic interactions buffered in the Stage Log are synthesized into persistent KB updates. This process is the core enabler of EngramTrace's non-parametric online learning.

**Stage Log and Encoding.** All query-answer pairs within a stage are recorded in the Stage Log $\mathcal{S}$, constituting an episodic memory buffer—a high-fidelity record of raw interaction data prior to semantic compression. The trace set $\mathcal{C}$ retrieved during each interaction is also recorded.

**Consolidation Trigger.** Consolidation is triggered by either (1) stage drift, or (2) the Stage Log reaching a maximum entry count (default: 15 entries).

**Synthesis and DOM Mutation.** The consolidation LLM receives $\mathcal{S}$ and the current KB document $\mathcal{K}$ and produces *surgical rewrite mutations*: minimal, targeted DOM modifications integrating new knowledge without disrupting existing structure. Mutations take three forms: node insertion (new element as child of appropriate parent), node update (existing node content revised), and node deletion (redundant or superseded node removed).

A **Root Protection** constraint prohibits the LLM from mutating root-level nodes of the KB tree. This prevents catastrophic structural rewrites that could destabilize the global knowledge organization, while allowing peripheral knowledge to evolve freely. Following mutation, hierarchical embeddings for all affected nodes and their descendants are recomputed and updated in the embedding index.

### 4.6 Homeostatic Day System

Over extended deployment, incremental consolidation accumulates structural drift: redundant nodes proliferate, the hierarchy misaligns with evolved knowledge content, and recent information may be structurally buried. The Homeostatic Day System addresses this through a periodic holistic restructuring pass triggered by inactivity exceeding 4000 seconds ($\approx$67 minutes), treated as a heuristic signal that the current continuous conversation has concluded.

During the Day Change, the entire KB is passed to the LLM for three operations: **Restructuring** (realign DOM hierarchy to better reflect current knowledge organization), **Reorganization** (promote recently integrated knowledge to structurally prominent positions), and **Deduplication** (merge redundant semantic fragments). The result is a new version of $\mathcal{K}$ with fully recomputed hierarchical embeddings. The Root Protection constraint applies to this operation as well.

## 5 The DOM as a Virtual Graph

### 5.1 Graph-Theoretic Properties

The parsed DOM constitutes a rooted ordered tree $\mathcal{T} = (\mathcal{N}, \mathcal{E}_{\text{parent-child}})$. Richer graph-theoretic structure emerges when considering the full set of semantic relationships extractable from the DOM:

- **Hierarchical edges** $\mathcal{E}_h$: Encode contextual containment—a paragraph within a section is semantically contextualized by that section, analogous to an *is-elaborated-by* relation in a KG.

- **Sibling edges** $\mathcal{E}_s$: Nodes sharing a common parent encode conceptual adjacency, membership in the same category or sequential components of a unified argument.
- **Embedding-derived synonym edges** $\mathcal{E}_e$: Implicit edges between nodes whose hierarchical embeddings exceed cosine similarity threshold $\tau$, analogous to HippoRAG's [3] synonymy edges.

The full virtual graph $G = (\mathcal{N}, \mathcal{E}_h \cup \mathcal{E}_s \cup \mathcal{E}_e)$ provides associative structure analogous to an explicit KG, derived implicitly from HTML structure and embedding similarity without requiring a graph database.

### 5.2 Advantages Over Explicit Graph Databases

**LLM-Native Representation.** Transformer models are pre-trained on corpora containing substantial HTML [6]. LLMs process HTML directly through the same representational machinery they apply to any structured document. Explicit KGs (RDF triples, Neo4j property graphs) must be linearized into text for LLM consumption, introducing a translation step that may distort structural information.

**Schema Freedom.** Explicit KGs impose an entity-relation-entity triple schema ill-suited to procedural knowledge, narrative context, multi-sentence arguments, or hierarchical taxonomies. The HTML DOM imposes no schema beyond parent-child tree structure, enabling representation of arbitrarily complex knowledge structures.

**Native Hierarchy.** HTML's heading structure (`<h1>` through `<h6>`) and block nesting provide semantically rich hierarchical organization that maps naturally onto the topic-subtopic-detail structure of most knowledge domains, directly enabling the hierarchical embedding scheme.

**Infrastructure Simplicity.** An explicit KG requires a dedicated graph database, query language (SPARQL, Cypher), and graph processing runtime. EngramTrace requires only a file system, BeautifulSoup4 for DOM parsing, and `numpy` for embedding storage.

**Human Interpretability.** An HTML KB can be read and edited by a human operator in any web browser. A Neo4j property graph requires specialized tooling. Interpretability enables direct inspection, correction, and validation by domain experts—important for trust and debugging.

**Batch Processing of Inter-Node Connectivity.** Since nodes do not exist as isolated entities but rather as textual content embedded within a single overarching HTML structure, an LLM can process their linkages in a single request, covering both semantic relationships and the entire network of connections simultaneously. This enables global, large-scale connectivity synchronization without sequential node-by-node verification.

### 5.3 Limitations of the Virtual Graph Approach

The implicit graph structure does not support arbitrary relation types with explicit predicates—a triple such as (`protein_X`, `inhibits`, `enzyme_Y`) must be encoded as prose in HTML, losing machine-queryable precision. Multi-hop graph traversal queries also require explicit graph construction from the DOM at query time. The virtual graph approach is therefore best suited to knowledge domains where hierarchical organization and semantic similarity are the primary retrieval axes; explicit KGs remain preferable for domains requiring precise relational reasoning over typed edge predicates.

# 6 Evaluation

## 6.1 Benchmark: MuSiQue

We evaluate EngramTrace on the **MuSiQue** benchmark [7], a multi-hop question answering dataset requiring integration of information across multiple passages. MuSiQue is designed specifically to resist spurious single-hop shortcuts, making it a meaningful test of whether a retrieval system genuinely navigates associative knowledge paths rather than relying on surface-level lexical matching. This property aligns directly with EngramTrace's core design motivation: moving beyond isolated chunk retrieval toward context-aware, hierarchically structured knowledge navigation.

MuSiQue questions require 2–4 reasoning hops, with supporting passages spread across documents. The benchmark includes both answerable and unanswerable variants; we evaluate on the answerable subset following standard protocol.

## 6.2 Experimental Setup

**Models.** Generation is performed by Google Gemini Flash-Lite [14], and embeddings are computed by Gemini Embedding-001 (256-dimensional). Response validation is performed by GPT-4o [15] as an LLM judge rather than exact string matching. This choice is deliberate: MuSiQue answers are evaluated against free-form system responses that may express a correct answer in sentence form rather than as a bare phrase, making word-level exact match an inappropriately strict metric. GPT-4o-based evaluation has been shown to correlate strongly with human judgment in QA settings and is the same validation approach used by Zep [5] on LongMemEval [19].

**Dataset scale.** Evaluation was conducted on 1,000 elements drawn from the MuSiQue answerable subset. Full-dataset evaluation was not completed due to API cost constraints—all inference and embedding calls are made through commercial APIs, and the per-query cost of running the full pipeline at scale was prohibitive. Importantly, the 1,000-element subset was not curated to favor favorable outcomes: no modification was made to result data, data ordering, or result selection. The subset reflects the first 1,000 elements in standard dataset order.

## 6.3 Results

EngramTrace achieves a GPT-4o-judged accuracy of **0.8111 (81.11%)** on the 1,000-element MuSiQue subset.

This figure should be interpreted with the evaluation method in mind. The existing MuSiQue literature—including HippoRAG [3] and related graph-based systems—reports performance using strict Exact Match (EM) scoring against gold answer strings, which typically yields figures in the 15–22% range for competitive systems on this benchmark. EM scoring is a deliberately conservative metric: a response that correctly identifies the answer but expresses it in sentence form scores zero. The GPT-4o judge used here evaluates semantic correctness rather than string identity, making the two metrics not directly numerically comparable. Reporting both under a single table would misrepresent the relationship between the scores.

What the 81.11% figure does meaningfully reflect is that EngramTrace produces correct answers to over four in five MuSiQue multi-hop questions as assessed by a strong semantic judge—on a benchmark specifically engineered to defeat systems that cannot genuinely integrate knowledge across multiple reasoning hops. MuSiQue's anti-shortcut design ensures that this result cannot be attributed to surface-level lexical retrieval or single-passage lookup; the system must locate and synthesize information across the 2–4 supporting passages each question requires. That

the hierarchical DOM retrieval mechanism achieves this at 81% accuracy under semantic evaluation, using a lightweight generation model (Gemini Flash-Lite) rather than a frontier model, suggests that the structural design choices—hierarchical embedding summation, ancestral context inheritance, and dual-mode Ecphory retrieval—are doing meaningful work.

A rigorous apples-to-apples comparison against graph-based baselines under a unified evaluation protocol remains as future work, contingent on available compute budget.

# 7 Qualitative Analysis

## 7.1 Novel Concept Knowledge Base

A particularly illustrative use case involves EngramTrace as a collaborative knowledge base for research on concepts that do not exist in the LLM's pre-training data. Pre-trained LLMs exhibit systematic knowledge gaps for ideas developed after their training cutoff or in proprietary, unpublished work—precisely the most novel and valuable conceptual territory.

EngramTrace enables the user to progressively build a custom knowledge base through conversation, with the system consolidating new knowledge into the persistent HTML structure across sessions. This is exemplified by using EngramTrace to develop the EngramTrace architecture itself: discussions about the system's design, rationale, and implementation details—none of which exist in any pre-training corpus—are maintained persistently and retrieved accurately across sessions. As the KB grows, subsequent queries are answered by reference to accumulated domain knowledge rather than parametric memory alone. This pattern demonstrates a qualitative advantage not easily captured by standard benchmarks: the ability to serve as a personalized, growing knowledge base for genuinely novel intellectual work, where system value scales directly with domain novelty.

Another distinguishing feature of EngramTrace is the ability for humans to directly intervene and modify the knowledge base itself, addressing potential errors that may arise during the knowledge accumulation process.

## 7.2 Technical Documentation Assistant

A second deployment involves using EngramTrace as a conversational interface over a private technical knowledge base—internal documentation for a software platform or API. Standard approaches require either providing full documentation in the context window at each interaction (prohibitively expensive, often exceeding context limits) or naive chunked RAG (which loses structural relationships between documentation sections).

EngramTrace's hierarchical KB structure maps naturally onto technical documentation: API modules correspond to top-level sections, individual endpoints to subsections, and parameter descriptions to leaf nodes. The hierarchical embedding scheme preserves module-level context when retrieving endpoint-level details, enabling the system to answer questions like "what authentication method does the data export endpoint use?" by correctly associating the endpoint detail with its module-level security context—a query that fails in flat retrieval if the authentication context is described in a parent section rather than repeated at the endpoint level.

# 8 Discussion

## 8.1 Positioning Relative to Explicit Graph-Based Systems

EngramTrace occupies a distinct position relative to explicit graph-based systems. The primary trade-off is between representational precision and operational simplicity. Explicit KGs with typed edge predicates provide machine-queryable precision that HTML prose cannot match. However, for conversational memory—where knowledge is primarily hierarchically organized, contextually nuanced, and expressed in natural language—the DOM-as-virtual-graph provides sufficient associative structure with dramatically lower infrastructure requirements.

The hierarchical embedding scheme provides context-sensitive retrieval addressing the same underlying problem as PPR-based graph traversal: ensuring retrieved knowledge is understood in proper context rather than semantic isolation. Both approaches improve retrieval quality through structural context; EngramTrace does so through continuous embedding blending rather than discrete graph traversal, with the trade-off that associative paths are implicit rather than explicitly queryable.

## 8.2 Online Consolidation vs. Static Indexing

A critical distinction between EngramTrace and static RAG systems—including GraphRAG [4], which requires full re-indexing for updates—is continuous knowledge integration through online consolidation. The surgical rewrite approach with Root Protection enables targeted updates that do not disrupt existing knowledge structure, addressing the catastrophic forgetting concern that plagues parametric approaches. This makes EngramTrace particularly suited to deployment scenarios where the knowledge base evolves continuously during use.

## 8.3 Limitations

**Consolidation Fidelity.** The consolidation step, relying on an LLM to synthesize episodic interactions into structured KB updates, is subject to hallucination and omission. The degree of information loss introduced by this step is an open empirical question to be addressed in evaluation.

**Scalability.** As the HTML KB grows over extended deployment, the embedding index grows linearly with the number of nodes, increasing retrieval latency and memory footprint. The Day Change operation involves passing the entire KB to the LLM, which becomes infeasible as the KB exceeds the LLM's context window. Chunked or hierarchical Day Change strategies are needed for large-scale deployment.

**Schema-Free Ambiguity.** The schema-free HTML representation introduces ambiguity: there is no formal guarantee that semantically similar knowledge will be organized in structurally similar ways across different consolidation operations, potentially reducing retrieval recall for related facts in structurally divergent subtrees.

**Unoptimized Hyperparameters.** The blending coefficients $\alpha = 0.7$ and $\beta = 0.7$, the drift threshold $\theta_{\text{drift}} = 0.83$, and the search threshold $\theta_{\text{search}} = 0.80$ are currently set to working defaults without systematic empirical validation. Their optimal values are likely domain- and task-dependent.

## 8.4 Future Directions

- Adaptive blending coefficients selected per-query or per-stage based on task characteristics.

- Multi-modal knowledge through `<img>` and `<figure>` elements with LLM-generated descriptions.
- Multi-agent memory with coordination protocols for concurrent consolidation operations over a shared KB.

# 9 Conclusion

This report presented **EngramTrace**, a non-parametric long-term memory architecture for LLM agents that introduces the HTML Document Object Model as a substrate for knowledge representation, retrieval, and online update. The core insight—that transformer models natively comprehend HTML, that the DOM provides implicit hierarchical graph structure without dedicated infrastructure, and that HTML's mutability enables continuous knowledge integration—motivates a system simultaneously more capable than flat RAG, simpler to deploy than explicit graph databases, and more interpretable than either.

The three primary technical contributions—Hierarchical Embedding Summation, dual-mode Ecphory retrieval, and online consolidation—address the fundamental limitations of existing approaches: semantic isolation in flat retrieval, infrastructure complexity in graph-based retrieval, and static indexing in existing RAG systems. Preliminary evaluation on MuSiQue demonstrates that EngramTrace correctly answers over four in five multi-hop questions under semantic evaluation—on a benchmark specifically engineered to defeat surface-level retrieval—suggesting that the hierarchical DOM structure and Ecphory retrieval mechanism are doing meaningful work in navigating multi-hop associative knowledge paths.

More broadly, EngramTrace represents a design philosophy: the most effective AI memory systems are those that align closely with the representational competencies of the LLMs they augment, operate through the same structural intuitions LLMs developed during pre-training, and remain interpretable and correctable by human operators. The HTML DOM, born as a tool for human-readable web documents, proves to be a surprisingly natural foundation for machine-readable, LLM-native long-term memory.

The full implementation is available at: https://github.com/iamtaehyunpark/EngramTrace

## References


[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

[3] Jiménez Gutiérrez, B., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems*, 37.

[4] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv:2404.16130*.

[5] Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., & Chalef, D. (2025). Zep: A temporal knowledge graph architecture for agent memory. *arXiv:2501.13956*.

[6] Tan, J., Dou, Z., Wang, W., Wang, M., Chen, W., & Wen, J.-R. (2025). HtmlRAG: HTML is better than plain text for modeling retrieved knowledge in RAG systems. *Proceedings of the ACM Web Conference 2025 (WWW '25)*. https://doi.org/10.1145/3696410.3714546

[7] Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10, 539–554.

[8] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2024). MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*.

[9] Teyler, T. J., & Discenna, P. (1986). The hippocampal memory indexing theory. *Behavioral Neuroscience*, 100(2), 147–154.

[10] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of EMNLP 2020*, 6769–6781.

[11] Eichenbaum, H. (2000). A cortical–hippocampal system for declarative memory. *Nature Reviews Neuroscience*, 1, 41–50.

[12] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.

[13] Richardson, L. (2024). Beautiful Soup. https://www.crummy.com/software/BeautifulSoup/

[14] Google DeepMind. (2024). Gemini: A family of highly capable multimodal models. *Technical Report*.

[15] OpenAI. (2023). GPT-4 technical report. *arXiv:2303.08774*.

[16] Haveliwala, T. H. (2002). Topic-sensitive PageRank. *Proceedings of WWW 2002*, 517–526.

[17] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive abstractive processing for tree-organized retrieval. *arXiv:2401.18059*.

[18] Teyler, T. J., & Rudy, J. W. (2007). The hippocampal indexing theory and episodic memory: Updating the index. *Hippocampus*, 17(12), 1158–1169.

[19] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., & Yu, D. (2024). LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv:2410.10813*.

[20] Guo, Z., Xia, L., Yu, Y., Ao, T., & Huang, C. (2024). LightRAG: Simple and fast retrieval-augmented generation. *arXiv:2410.05779*.